NAME: KEERTHIKA S **ROLL NO: 231901024**

TITLE: EXP NO.: 10 TROUBLESHOOTING FABRIC APPLICATIONS

1. AIM

To identify, analyze, and resolve common issues encountered in Hyperledger Fabric applications by troubleshooting network connectivity, chaincode deployment, channel configuration, certificate authentication, and Docker container errors.

2. TOOLS / SOFTWARE REQUIRED

- Kali Linux / Ubuntu
- Docker
- Docker Compose
- Node.js (Version 18 or later)
- npm
- Git
- Hyperledger Fabric Samples
- Hyperledger Fabric Binaries
- Hyperledger Fabric Docker Images
- Visual Studio Code

3. HARDWARE REQUIREMENTS

- Processor: Intel Core i3 or above
- RAM: Minimum 8 GB
- Storage: Minimum 20 GB Free Disk Space
- Internet Connection

4. THEORY

Troubleshooting is an essential part of administering Hyperledger Fabric networks. Errors may occur due to incorrect configuration, network failures, certificate issues, chaincode deployment failures, or Docker container problems. Common troubleshooting techniques include: • Checking Docker container status • Viewing peer and orderer logs • Verifying channel membership • Checking chaincode installation • Validating MSP and certificates • Restarting Fabric services • Cleaning Docker resources Proper

troubleshooting minimizes downtime and ensures reliable blockchain application deployment.

5. PROCEDURE

Step 1: Navigate to the Fabric Test Network

```
cd ~/fabric-dev/fabric-samples/test-network
```

Step 2: Start the Hyperledger Fabric Network

```
./network.sh up createChannel -ca
```

Step 3: Verify Docker Containers

Check whether all required containers are running.

```
docker ps
```

Step 4: Check Peer Logs

Display the logs of Peer Organization 1.

```
docker logs peer0.org1.example.com
```

Step 5: Check Orderer Logs

```
docker logs orderer.example.com
```

Step 6: Verify Channel Membership

```
peer channel list
```

Step 7: Verify Installed Chaincode

```
peer lifecycle chaincode queryinstalled
```

Step 8: Troubleshoot Chaincode Deployment

Redeploy the chaincode if deployment has failed.

```
./network.sh deployCC
-ccl javascript
-ccn basic
-ccp ../asset-transfer-basic/chaincode-javascript
```

Step 9: Verify Chaincode Execution

Query the ledger to ensure that the chaincode is functioning correctly.

```
peer chaincode query
-C mychannel
-n basic
-c '{"Args":["GetAllAssets"]}'
```

Step 10: Check Docker Resource Usage

Monitor resource utilization.

```
docker stats
```

Step 11: Restart the Fabric Network

Stop and restart the network.

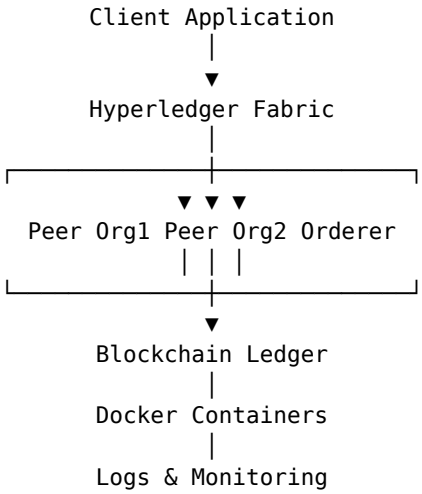
```
./network.sh down
./network.sh up createChannel -ca
```

Step 12: Clean Docker Resources (Optional)

Remove unused Docker resources.

```
docker system prune -f
```

6. ARCHITECTURE DIAGRAM



7. COMMON ERRORS AND SOLUTIONS

Error Message	Possible Cause	Solution
Docker daemon not running	Docker service stopped	sudo systemctl start docker
Peer connection failed	Peer container stopped	Restart Fabric network
Chaincode deployment failed	Incorrect chaincode path	Verify -ccp directory
Channel not found	Channel not created	Run ./network.sh createChannel
Access denied	Invalid MSP or certificates	Regenerate certificates
Chaincode not installed	Installation failed	Redeploy chaincode
TLS handshake failed	Incorrect TLS certificate	Verify TLS CA files
Port already in use	Another service using the port	Stop conflicting process

8. SUMMARY OF KEY COMMANDS

Purpose	Command
Navigate	`cd ~/fabric-dev/fabric-samples/test-network`
Start network	`./network.sh up createChannel -ca`
Check containers	`docker ps`
View logs	`docker logs peer0.org1.example.com`
View logs	`docker logs orderer.example.com`
Verify channel	`peer channel list`
Query ledger	`peer lifecycle chaincode queryinstalled`
Deploy chaincode	`./network.sh deployCC`
Query ledger	`peer chaincode query`
Monitor resources	`docker stats`
Stop network	`./network.sh down`
Command	`docker system prune -f`

9. OUTPUT

The terminal output below is rendered from the output blocks supplied in the source experiment.

Output 1

```
Experiment 10 - Fabric Output

$ Fabric Network Started Successfully
Peer Verified Successfully
Orderer Verified Successfully
Channel Verified Successfully
Chaincode Installed Successfully
Application Executed Successfully
No Errors Detected
Fabric Network Running Normally
```

Fig. 10.1 – Fabric output corresponding to the source experiment output.

10. RESULT

The Hyperledger Fabric application was successfully analyzed and common operational issues were identified and resolved using Docker commands, Fabric CLI tools, and log analysis. The experiment demonstrated effective troubleshooting techniques for network connectivity, chaincode deployment, channel configuration, certificate management, and resource monitoring.